UNIT 1 — Introduction to AI and Intelligent Agents

Subject: Artificial Intelligence

Branch: Computer Engineering | Sem: V | SPPU 2024 Pattern

SYLLABUS TOPICS:

1.1 Introduction to Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science that deals with building machines capable of performing tasks that normally require human intelligence, such as reasoning, learning, perception and decision making. An AI system takes percept’s from its environment and produces actions that improve its performance on a given task.

Introduction / Need:

Human beings solve everyday problems using reasoning, past experience and common sense. Machines, on the other hand, only follow instructions written by a programmer.

AI tries to close this gap by giving machines the ability to reason, learn from data and adapt to new situations, instead of just executing a fixed program written in advance.

1.1.1 What is Artificial Intelligence:

AI can be understood as making computers do things that, if done by a human, would be called intelligent. This includes tasks like understanding language, recognising images and playing strategy games.

AI is not a single technique. It is an umbrella field that covers search algorithms, logic based reasoning, probability, machine learning and robotics, all aimed at building intelligent behaviour.

Sr. No.	Approach	Focus	Example
1	Thinking Humanly	Modelling human thought process	Cognitive modelling, simulating human reasoning steps
2	Thinking Rationally	Modelling logical, correct reasoning	Laws of thought, syllogisms, logic programming
3	Acting Humanly	Behaving indistinguishably from a human	Turing Test based chatbots
4	Acting Rationally	Choosing the best action for a given goal	Rational agents, modern AI systems
5	Hybrid View	Combines rational thought with rational action	Most present day AI systems (e.g. Google Assistant)

1.1.2 The Turing Test and Acting Humanly

Alan Turing proposed the Turing Test (imitation game where a human judge talks to a machine and a human through text and tries to tell them apart) in 1950 as a practical way to judge machine intelligence.

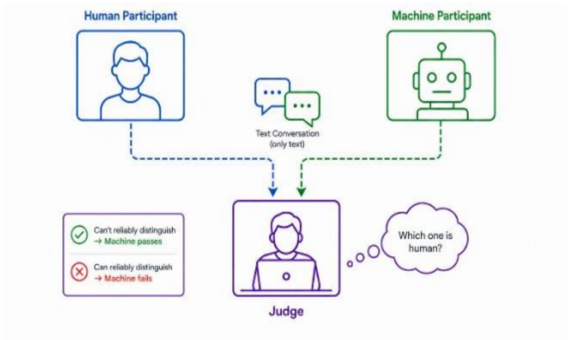


Fig 1.1 — Turing Test setup with human judge, human respondent and machine

If the judge cannot reliably tell which one is the machine, the machine is said to have passed the test and shown human-like intelligence.

1.1.3 AI versus Human Intelligence and Machine Learning

Human intelligence is general purpose. A person can cook food, drive a car and write a poem using the same brain, without being reprogrammed for each task.

Most present day AI is narrow AI (designed and trained for one specific task, unable to perform outside that task) such as a spam filter or a face recognition system.

1.1.4 Why Study Artificial Intelligence

AI is used almost everywhere today, from recommendation systems on shopping sites to medical diagnosis tools, so understanding it is essential for every computer engineer.

Studying AI gives students the tools to automate repetitive decision making tasks, build smarter software products and analyse large amounts of data efficiently.

Advantages:

- Automates repetitive decisions — AI systems can make routine decisions faster than manual human review, saving time in industries like banking.
- Handles large data volumes — AI can process and find patterns in datasets that are too large for manual human analysis.
- Improves accuracy — well trained AI models reduce human error in tasks like medical image screening.
- Enables 24x7 operation — AI powered systems such as chatbots can serve users continuously without fatigue.

Disadvantages:

- High development cost — building and training accurate AI models requires significant computing power and skilled manpower.
- Data dependency — AI systems need large amounts of good quality data, and perform poorly when data is limited or biased.
- Lack of common sense — AI systems can fail on situations outside their training data, unlike humans who use general reasoning.

Applications:

- Google Search uses AI ranking algorithms to order billions of web pages by relevance.
- Amazon uses AI based recommendation engines to suggest products to shoppers.
- Tesla Autopilot uses AI perception and planning modules for semi-autonomous driving.
- IBM Watson for Oncology uses AI to assist doctors in cancer treatment planning.

KEY POINT BOX

- AI is defined along two axes: thought vs action, and human-like vs rational — SPPU frequently asks to name and explain all four views.
- The Turing Test tests acting humanly, not rational behaviour — do not confuse the two in exam answers.
- Narrow AI (task specific) is what exists today; General AI (human level, multi-task) is still a research goal.
- Students often confuse AI, Machine Learning and Deep Learning — remember ML and Deep Learning are subsets of AI, not the same thing.
- The four approaches to AI (thinking humanly, thinking rationally, acting humanly, acting rationally) is a guaranteed 5 or 6 mark question.

EXPECTED QUESTIONS BOX

Q1. Explain the concept of Artificial Intelligence with its four defining approaches. (6 marks)

Q2. Describe the Turing Test with a neat diagram. (5 marks)

Q3. Differentiate between Artificial Intelligence, Machine Learning and Deep Learning. (6 marks)

Q4. Explain why acting rationally is considered a better standard for AI than acting humanly. (5 marks)

Q5. Discuss the need and importance of studying Artificial Intelligence with real world examples. (8 marks)

1.2 Foundations of Artificial Intelligence

Definition:
The foundations of AI refer to the different academic disciplines — such as philosophy, mathematics, economics, neuroscience, psychology, linguistics and computer engineering — whose ideas, tools and techniques were combined to create the field of Artificial Intelligence.

Introduction / Need:
AI did not appear out of nowhere. It borrowed ideas from many older fields that had already studied thinking, logic and behaviour for centuries.

Knowing these foundations helps students understand why AI uses tools like logic (from philosophy), probability (from mathematics) and neural networks (from neuroscience).

1.2.1 Contributing Disciplines of AI

Explanation:
Philosophy contributed ideas about logic, mind and reasoning, including questions like whether a machine can truly think.

Mathematics gave AI the formal tools of logic, probability, statistics and algorithms needed to represent uncertainty and computation.

Sr. No.	Discipline	Contribution to AI	Example Concept Borrowed
1	Philosophy	Logic, foundations of reasoning and mind	Formal logic, rules of valid inference
2	Mathematics	Formal representation and computation	Probability theory, algorithms, logic
3	Economics	Rational decision making under uncertainty	Game theory, utility theory
4	Neuroscience	Working of biological neurons	Artificial neural networks
5	Psychology	Human thought and behaviour patterns	Cognitive modelling

Advantages:
- Multi-disciplinary base — combining many fields gives AI a richer toolbox than any single discipline could provide.
- Strong theoretical grounding — mathematics and logic give AI systems provable and testable behaviour.
- Biological inspiration — neuroscience based models like neural networks have proven extremely powerful in practice.

Disadvantages:
- Complex learning curve — students need basic exposure to logic, probability and cognitive science to fully understand AI.
- Discipline gaps — insights from one field (e.g. psychology) do not always translate cleanly into working algorithms.

Applications:

- Game theory from Economics is used in multi-agent bidding systems like Google Ads auctions.
- Neural network models from Neuroscience power image recognition in Facebook photo tagging.
- Logic based reasoning from Philosophy is used in expert systems like MYCIN for medical diagnosis.

KEY POINT BOX

KEY POINTS — Foundations of Artificial Intelligence

- Memorise at least five contributing disciplines with one contribution each — this is a standard SPPU question.
- Philosophy raised the question of thinking machines; Mathematics gave AI its formal tools.
- Neuroscience inspired neural networks; Economics inspired rational agent theory and game theory.
- Do not confuse contributing disciplines with the History of AI timeline — they are different questions.
- Linguistics is specifically linked to Natural Language Processing in most SPPU answer keys.

EXPECTED QUESTIONS BOX

SPPU EXPECTED QUESTIONS — Foundations of Artificial Intelligence

Q1. Explain any five foundations (contributing disciplines) of Artificial Intelligence. (6 marks)

Q2. Describe the role of Mathematics and Philosophy in shaping Artificial Intelligence. (5 marks)

Q3. How did Neuroscience influence the development of Artificial Neural Networks. (5 marks)

Q4. Discuss the contribution of Economics and Psychology to AI with examples. (6 marks)

1.3 History of Artificial Intelligence

Definition:

The history of Artificial Intelligence refers to the timeline of key events, from its formal beginning at the 1956 Dartmouth conference through periods of rapid growth, funding cuts known as AI winters, and the recent resurgence driven by machine learning and deep learning.

Introduction / Need:

AI as a field has gone through repeated cycles of excitement and disappointment over the last 70 years.

Studying this history helps engineers understand why certain techniques (like neural networks) fell out of favour and later became dominant again with more data and computing power.

1.3.1 Key Phases in AI History

Explanation:

The gestation period (1943 to 1955) saw the first mathematical model of an artificial neuron by McCulloch and Pitts, laying the groundwork for later neural network research.

AI was formally born in 1956 at the Dartmouth Conference, where John McCarthy first coined the term 'Artificial Intelligence' and researchers proposed that machine intelligence could be precisely described and simulated.

Sr. No.	Period	Milestone Event	Key Contributor / System
1	1943	First artificial neuron model	McCulloch and Pitts
2	1949	Hebbian Learning Rule introduced	Donald Hebb
3	1950	Turing Test proposed	Alan Turing
4	1956	Term 'Artificial Intelligence' coined	John McCarthy, Dartmouth Conference
5	1958	LISP programming language developed for AI	John McCarthy
6	1959–1966	Early AI systems developed (GPS, Geometry Theorem Prover, Advice Taker, ELIZA)	Newell & Simon, Gelernter, McCarthy, Weizenbaum
7	1974–1980	First AI Winter due to reduced funding and unmet expectations	Lighthill Report (1973)
8	1980	Rise of Expert Systems	DENDRAL, MYCIN
9	2012–Present	Deep Learning and Generative AI revolution	Large Language Models (LLMs)

Advantages:

- Shows real progress — the AI winters prove AI has survived doubt and consistently improved with new techniques.
- Prevents repeated mistakes — knowing past over-hype helps current researchers set realistic expectations.
- Explains technique choice — history explains why symbolic AI dominated early and statistical AI dominates now.

Disadvantages:

- Overwhelming detail — the full history has dozens of events, which can confuse students trying to memorise everything.
- Repetition risk of past failures — same over-promising pattern could occur again with current generative AI hype.

Applications:

- IBM Deep Blue's 1997 chess win is cited in every discussion of search based game playing AI.
- MYCIN's rule based diagnosis approach directly inspired modern clinical decision support systems.
- AlexNet's 2012 win is referenced in every deep learning course as the beginning of the deep learning era.

KEY POINT BOX
KEY POINTS — History of Artificial Intelligence • 1956 Dartmouth Conference is the single most quoted fact in AI history — remember John McCarthy coined the term. • There were two AI winters — mid 1970s and late 1980s — know the reason for each. • Expert systems (MYCIN, DENDRAL) mark the shift from theory to real commercial AI use. • 2012 AlexNet result is the turning point that started the current deep learning boom. • SPPU commonly asks for a 'timeline' style answer — always answer in a table or point-wise year format.

EXPECTED QUESTIONS BOX
SPPU EXPECTED QUESTIONS — History of Artificial Intelligence **Q1.** Explain the major milestones in the history of Artificial Intelligence with a timeline. (8 marks) **Q2.** What is an AI winter? Explain the reasons behind the first and second AI winter. (6 marks) **Q3.** Describe the significance of the Dartmouth Conference of 1956. (5 marks) **Q4.** Explain the rise of expert systems in the 1980s with examples. (5 marks)

1.4 Limits of AI

Limits of AI refer to the fundamental technical, computational and conceptual boundaries that prevent current Artificial Intelligence systems from achieving true human-like general intelligence, common sense reasoning and full understanding of context.

Introduction / Need:
Despite impressive results, AI systems today are still narrow and can fail badly on tasks that a five year old child would find trivial, such as understanding sarcasm.

Understanding these limits is important for engineers so that AI systems are deployed responsibly, especially in safety critical areas like healthcare and self driving cars.

1.4.1 Technical and Conceptual Limits:

Lack of common sense reasoning is a major limit — AI models struggle with everyday facts about the world that humans learn naturally, such as knowing that water makes things wet.

Data dependency is another limit — most AI models need huge, carefully labelled datasets, and perform poorly when data is scarce, biased or noisy.

Sr. No.	Type of Limit	Description	Example
1	Common sense limit	No built-in real-world knowledge	AI failing to know ice melts in sun
2	Data limit	Needs huge labelled datasets	Poor accuracy for rare diseases with little data
3	Generalisation limit	Fails outside training distribution	Self-driving car confused by snow it never saw
4	Explainability limit	Cannot explain its own decision	Deep learning model denying a loan with no reason
5	Philosophical limit	Debated true understanding vs simulation	Chinese Room argument by John Searle

Advantages:

- Encourages safe design — knowing the limits pushes engineers to add human oversight in critical AI systems.
- Guides research direction — limits like poor generalisation are now driving research into more robust AI models.
- Prevents over-trust — awareness of limits stops organisations from blindly trusting AI decisions.

Disadvantages:

- Slower adoption — highlighting limits can make businesses overly cautious and delay useful AI adoption.
- Public confusion — mixing technical limits with philosophical debates like the Chinese Room can mislead non-experts.

Applications:

- Autonomous vehicle companies like Waymo actively test edge cases to address the generalisation limit before deployment.
- Healthcare AI regulators require explainability, pushing hospitals to use interpretable models over black box ones.
- Financial institutions add human review for AI credit scoring decisions because of the explainability limit.

KEY POINT BOX

KEY POINTS — Limits of AI

- Common sense reasoning failure is the most exam-favoured example of a limit of AI.
- The Chinese Room argument by John Searle is the standard philosophical limit asked in SPPU papers.
- Black box / explainability problem is heavily tested in the context of deep learning limits.
- Do not confuse Limits of AI with Ethics of AI — limits are technical/conceptual, ethics is about right/wrong use.
- Data dependency and poor generalisation are the two most common technical limits cited in answers.

EXPECTED QUESTIONS BOX

SPPU EXPECTED QUESTIONS — Limits of AI

Q1. Explain the major technical limits of Artificial Intelligence with examples. (6 marks)

Q2. What is the Chinese Room argument? Explain its significance for AI. (5 marks)

Q3. Discuss the black box problem in AI and its impact on trust. (5 marks)

Q4. Why does AI struggle with common sense reasoning? Explain with an example. (6 marks)

1.5 Ethics of AI

Definition:

Ethics of AI is the study of moral principles and guidelines that govern the responsible design, development and deployment of Artificial Intelligence systems, ensuring they are fair, transparent, safe, and respect human rights and privacy.

Introduction / Need:

As AI systems start making decisions that affect real people, such as loan approvals or hiring, the fairness and correctness of those decisions becomes a serious social concern.

Ethics of AI is now a required topic for engineers because poorly designed AI can unintentionally discriminate, invade privacy, or cause harm at a massive scale.

1.5.1 Core Ethical Issues in AI

Bias and fairness (AI systems reflecting or amplifying unfair patterns present in their training data) is a top concern, such as facial recognition systems performing worse on certain skin tones.

Privacy concerns arise because AI systems often need large amounts of personal data, raising the risk of surveillance and misuse of that data.

Sr. No.	Ethical Issue	Description	Real-World Example
1	Bias and Fairness	Model reflects unfair patterns in training data	Biased hiring AI at Amazon (discontinued in 2018)
2	Privacy	Excess collection or misuse of personal data	Facial recognition surveillance concerns
3	Accountability	Unclear responsibility for AI errors	Self-driving car accident liability debates
4	Job Displacement	Automation replacing human roles	Automated customer support reducing call centre jobs
5	Safety and Alignment	AI goals not matching human intentions	Reinforcement learning agents exploiting reward loopholes

Advantages:

- Builds public trust — ethical AI practices make users more comfortable adopting AI powered products.
- Reduces legal risk — following ethical guidelines helps companies avoid lawsuits related to discrimination or privacy breaches.
- Improves fairness — bias testing during development leads to more fair outcomes across different user groups.

Disadvantages:

- No single standard — different countries and companies define AI ethics differently, causing inconsistency.
- Slower development — extra fairness and bias checks can increase project timelines and cost.

Applications:

- The EU AI Act uses ethical risk categories to regulate high risk AI systems across Europe.
- Google's AI Principles publicly commit to avoiding harmful bias in their AI products.
- IBM's AI Fairness 360 toolkit is used by developers to test and reduce bias in machine learning models.

KEY POINT BOX
• Bias and Fairness is the single most commonly asked ethics sub-topic in SPPU exams. • Always separate Ethics of AI (should we do it) from Limits of AI (can we do it) in answers. • AI Safety and Alignment is a growing exam focus given recent generative AI growth. • Real examples like Amazon's biased hiring tool score extra marks in descriptive answers. • Accountability questions often link to real world cases like autonomous vehicle accidents.

EXPECTED QUESTIONS BOX
Q1. Explain the major ethical issues in Artificial Intelligence with examples. (6 marks)
Q2. What is bias in AI systems? Explain with a real-world case study. (5 marks)
Q3. Discuss the importance of accountability and transparency in AI systems. (5 marks)
Q4. Explain the concept of AI safety and alignment. (6 marks)

1.6 Future of AI

The future of Artificial Intelligence refers to expected upcoming trends and directions of AI development, including progress toward Artificial General Intelligence, wider use of generative AI, stronger regulation, and deeper collaboration between humans and intelligent systems.

Introduction / Need:

AI is evolving faster than almost any other technology field, moving from narrow task specific systems to increasingly general and creative systems within a few years.

Engineers today must not only understand current AI techniques but also anticipate emerging trends to stay relevant in their careers.

1.6.1 Emerging Directions in AI

Artificial General Intelligence (AGI, a hypothetical AI with human level intelligence across all tasks, not just one) remains a long term research goal actively pursued by major AI labs.

Generative AI (AI systems that create new content such as text, images or code, rather than only classifying data) such as ChatGPT and image generators is expanding rapidly into everyday tools.

Sr. No.	Future Trend	Description	Example System
1	AGI Research	Pursuit of general, human-level intelligence	Long term goal of labs like DeepMind, OpenAI
2	Generative AI	AI creating new text, images, code	ChatGPT, Midjourney, GitHub Copilot
3	Edge AI	AI running locally on devices	On-device AI in modern smartphones
4	AI Regulation	Legal frameworks controlling AI risk	EU AI Act, national AI policies
5	Human-AI Collaboration	AI as assistant, not replacement	AI co-pilots in coding and writing tools

Advantages:

- Career growth — understanding future AI trends helps engineering students choose relevant specialisations.
- Productivity boost — generative and co-pilot style AI tools are already increasing developer and writer productivity.
- Accessibility — edge AI brings intelligent features to low connectivity regions without depending on cloud servers.

Disadvantages:

- Uncertain timeline — nobody can accurately predict when, or if, true AGI will be achieved.
- Hype versus reality gap — many future AI predictions are exaggerated by media beyond what is technically realistic.

Applications:

- GitHub Copilot already shows the future trend of AI assisted software development.
- ChatGPT and similar large language models represent the current wave of generative AI adoption.
- Apple's on-device AI features in iPhones demonstrate the growth of Edge AI.

KEY POINT BOX

- AGI is still theoretical — do not present it as an achieved milestone in exam answers.
- Generative AI (ChatGPT-like systems) is the most exam-relevant current future trend for 2024 pattern papers.
- Edge AI is linked to privacy and low-latency benefits — a common comparison point.
- AI regulation trends (EU AI Act) show the shift from technical development to policy control.
- Future of AI questions often ask for both trends and challenges — always cover both sides.

EXPECTED QUESTIONS BOX

Q1. Discuss the future trends in Artificial Intelligence. (6 marks)

Q2. What is Artificial General Intelligence? How is it different from Narrow AI. (5 marks)

Q3. Explain the role of Generative AI in shaping the future of technology. (5 marks)

Q4. Discuss the growing importance of AI regulation with examples. (6 marks)

1.7 AI Components

AI Components are the fundamental functional building blocks — knowledge representation, reasoning, learning, perception, planning and communication — that together allow an intelligent system to sense its environment, process information, and act intelligently to achieve its goals.

Introduction / Need:

Every working AI system, no matter how simple or advanced, is built from a small set of core capabilities working together.

Understanding these components separately makes it easier to design, debug and improve any AI system, since each component can be studied and tested on its own.

1.7.1 Knowledge Representation and Reasoning

Knowledge Representation (the method of storing facts about the world in a form a computer can process) allows an AI system to store information as rules, semantic networks, or logical statements.

Reasoning (the process of using stored knowledge to derive new facts or conclusions) allows the AI to answer questions or make decisions it was not explicitly told the answer to.

1.7.2 Learning

Learning (the ability of an AI system to improve its performance automatically from experience or data, without being explicitly reprogrammed) is central to almost all modern AI systems.

Machine learning methods include supervised learning (learning from labelled examples), unsupervised learning (finding patterns in unlabelled data), and reinforcement learning (learning through trial, error and reward).

1.7.3 Perception, Planning and Communication

Perception (converting raw sensor data such as images or sound into meaningful information) enables an AI system to understand its environment, as used in computer vision and speech recognition.

Planning (deciding a sequence of actions in advance to achieve a goal) allows an AI agent to work toward long term objectives rather than just react to immediate inputs.

Sr. No.	Component	Function	Example Technology
1	Knowledge Representation	Stores facts in usable form	Semantic networks, ontologies
2	Reasoning	Derives new conclusions from facts	Rule based inference engines
3	Learning	Improves performance from data/experience	Neural networks, decision trees
4	Perception	Interprets sensory input	Computer vision, speech recognition
5	Planning	Decides action sequence toward a goal	A* search, STRIPS planner

Advantages:

- Modular design — building AI from separate components makes systems easier to design and debug.
- Reusability — a perception module built for one project, like an image classifier, can often be reused in another.
- Clear specialisation — researchers can focus deeply on one component, such as reasoning, without needing expertise in all of them.
- Better system understanding — breaking AI into components helps engineers explain and justify system behaviour.

Disadvantages:

- Integration complexity — combining separately built components into one working system is technically difficult.
- Performance bottlenecks — a weak component, like poor perception, can limit the accuracy of the whole system.
- Increased resource need — running all components together (perception, planning, learning) needs significant computing power.

Applications:

- Autonomous cars like Tesla combine perception (cameras), planning (path finding) and learning (driving policy) together.
- Amazon Alexa combines perception (speech recognition), NLP (understanding commands) and reasoning to respond to users.
- IBM Watson combines knowledge representation and reasoning for enterprise question answering systems.
- Google Photos uses the perception component for automatic face and object tagging in images.

KEY POINT BOX
• SPPU commonly asks to 'list and explain the components of AI' — always cover at least five to six components. • Perception handles input, Planning and Reasoning handle decision making, Learning improves the system over time. • NLP is the practical technology behind the Communication component — link them clearly in answers. • Knowledge Representation and Reasoning are paired concepts and are frequently asked together. • Do not confuse AI Components with AI Architectures — components are functions, architectures are how they are structured.
EXPECTED QUESTIONS BOX
Q1. Explain the major components of an Artificial Intelligence system. (8 marks) **Q2.** Differentiate between Knowledge Representation and Reasoning as AI components. (5 marks) **Q3.** Explain the role of Learning and Perception in modern AI systems. (6 marks) **Q4.** Describe how Planning and Communication contribute to an AI system, with examples. (6 marks)

1.8 AI Architectures

AI Architectures define the overall structural design used to organise the components of an intelligent system, determining how knowledge is stored, how decisions are made, and how the system's parts (symbolic, connectionist, or hybrid) interact with one another.

Introduction / Need:

Just as a building needs an architectural plan before construction, an AI system needs an architecture that decides how its reasoning, learning and perception components fit together.

Choosing the right architecture affects how well an AI system scales, how explainable it is, and how well it handles uncertainty in the real world.

1.8.1 Symbolic (Rule-Based) Architecture

Symbolic architecture (representing knowledge using explicit symbols, rules and logic) was the dominant style of classical AI, used in early expert systems.

It works well for well-defined, logic based problems, and produces explainable decisions since every conclusion can be traced back to a rule.

1.8.2 Connectionist (Neural Network) Architecture

Connectionist architecture (representing knowledge as weighted connections between artificial neurons, learned automatically from data) underlies modern deep learning systems.

It excels at pattern recognition tasks like image and speech recognition, where rules would be nearly impossible to hand write.

1.8.3 Hybrid and Subsumption Architectures

Hybrid architecture (combining symbolic reasoning with connectionist learning in one system) tries to get the explainability of rules and the pattern recognition power of neural networks together.

Subsumption architecture (a layered robotic control design where simple behaviour layers are built first, and higher layers can override lower ones) was proposed by Rodney Brooks for reactive robots that respond quickly without heavy reasoning.

Sr. No.	Architecture Type	Knowledge Style	Strength	Weakness
1	Symbolic	Explicit rules and logic	Explainable, precise reasoning	Poor with noisy real-world data
2	Connectionist	Learned neural weights	Excellent pattern recognition	Hard to explain decisions
3	Hybrid	Rules plus learned models	Balances accuracy and explainability	Complex to design and integrate
4	Subsumption	Layered reactive behaviours	Fast, robust real-time robot control	Limited high-level reasoning
5	Cognitive (SOAR/ACT-R)	Unified memory and rule system	Models general human-like cognition	Very complex, mainly research use

Advantages:

- Structured design — a clear architecture makes large AI systems easier to build, test and maintain.
- Task-specific optimisation — choosing symbolic vs connectionist lets engineers match the architecture to the problem type.
- Better scalability — layered architectures like subsumption scale well for real time robotic control.
- Improved explainability options — symbolic and hybrid architectures allow decisions to be traced and justified.

Disadvantages:

- Design complexity — choosing and integrating the right architecture requires deep expertise and experimentation.
- Trade-off problem — no single architecture gives both full explainability and top pattern-recognition performance.
- High resource needs — connectionist and hybrid architectures typically need heavy computing power such as GPUs.

Applications:

- MYCIN, an early medical expert system, used a symbolic rule based architecture for diagnosis.
- Google's image recognition systems use connectionist (deep neural network) architecture.
- IBM Watson used a hybrid architecture combining symbolic knowledge bases with statistical NLP methods.
- iRobot Roomba style robots use subsumption architecture for simple, fast obstacle avoidance behaviour.

KEY POINT BOX

- Symbolic = rules and logic, explainable; Connectionist = neural weights, learned, less explainable — this contrast is a common SPPU question.
- Subsumption architecture (Rodney Brooks) is specifically linked to fast, reactive robot control.
- Hybrid architecture is the practical modern answer combining the strengths of both symbolic and connectionist approaches.
- SOAR and ACT-R are the two most quoted cognitive architectures in SPPU answer keys.
- Do not confuse AI Architecture (structural design) with Agent Architecture used later in agent types — link them when explaining agent structure.

EXPECTED QUESTIONS BOX

Q1. Explain Symbolic and Connectionist AI architectures with a comparison. (6 marks)
Q2. What is Subsumption Architecture? Explain with its advantages. (5 marks)
Q3. Describe Hybrid AI architecture and why it is preferred in modern systems. (5 marks)
Q4. Explain any two cognitive architectures used in Artificial Intelligence. (6 marks)

1.9 Intelligent Agents

An intelligent agent is anything that can perceive its environment through sensors and act upon that environment through actuators, in order to achieve a specific goal or maximise its performance measure. It maps percept sequences to actions using an internal decision process.

Introduction / Need:
The concept of an agent gives AI a unified way to describe very different systems, from a simple thermostat to a complex self driving car, using the same basic sense-think-act cycle.

This unit treats agents as the central organising idea of AI: instead of studying isolated algorithms, we study how a complete agent perceives, decides and acts inside an environment.

1.9.1 Agent Function and Agent Program

The agent function (a mathematical mapping from every possible percept sequence the agent has seen so far, to an action) fully describes an agent's behaviour in the abstract.

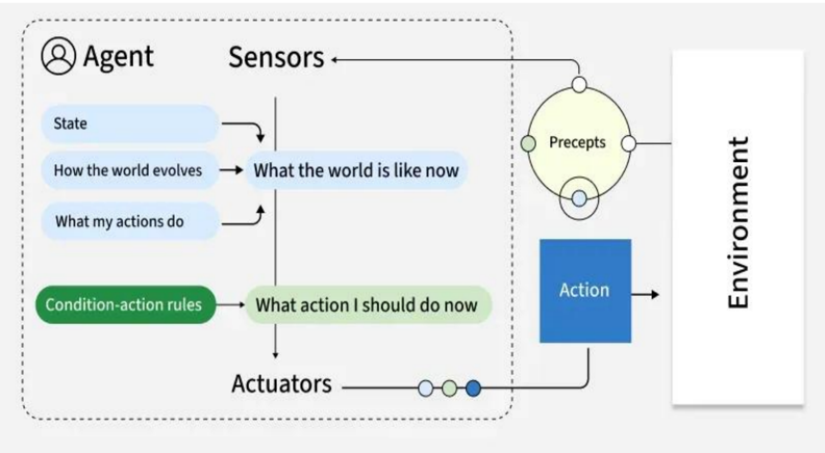
The agent program (the concrete implementation, running on the agent's physical hardware or software, that computes the agent function) is what actually gets built and executed.

Sr. No.	Aspect	Agent Function	Agent Program
1	Nature	Abstract mathematical mapping	Concrete running implementation
2	Input	Complete percept sequence	Current percept (plus internal state)
3	Size	Can be infinitely large	Compact, finite code
4	Where it exists	Conceptual / theoretical	Executes on real agent hardware
5	Example	Table mapping all vacuum world histories to actions	C program controlling a Roomba robot

1.9.2 Percepts, Actions, Sensors and Actuators

Percept (a single unit of perceptual input received by the agent at any instant) is what the agent senses at one moment, such as a camera frame or a temperature reading.

Percept sequence (the complete history of everything the agent has perceived so far) matters because an agent's action can depend on past percepts, not just the current one.



1.9.3 PEAS Framework

PEAS (a framework to specify a task environment by describing its Performance measure, Environment, Actuators and Sensors) is the standard first step in designing any intelligent agent.

Performance measure defines what counts as success for the agent, such as safety, speed, and passenger comfort for a taxi driving agent.

Sr. No.	PEAS Element	Meaning	Example (Automated Taxi)
1	Performance measure	Criteria for judging success	Safety, speed, legal driving, profit, comfort
2	Environment	The surroundings agent operates in	Roads, traffic, pedestrians, weather
3	Actuators	Components agent acts with	Steering, accelerator, brake, horn, display
4	Sensors	Components agent perceives with	Cameras, GPS, speedometer, sonar
5	Example task 2	Vacuum cleaner PEAS	Cleanliness (PM), Room (E), Wheels/Suction (A), Dirt sensor (S)

Advantages:

- Unified framework — the agent concept lets very different AI systems (robots, chatbots, game AI) be studied under one theory.
- Clear design starting point — PEAS forces engineers to precisely define the problem before coding an AI system.
- Supports comparison — describing systems as agents makes it easy to compare their performance and environment assumptions.
- Encourages modular thinking — separating sensors, actuators and decision logic leads to cleaner system design.
- Scales from simple to complex — the same agent model applies to a thermostat and to a self driving car.

Disadvantages:

- Abstraction overhead — beginners often find the agent function/program distinction too theoretical at first.
- PEAS can be incomplete — some real world systems have overlapping or hard to separate performance measures.
- Doesn't specify internal design — PEAS says what an agent should do, not how to build its internal reasoning.
- Environment complexity underestimated — real environments (like public roads) are far more complex than a PEAS table suggests.

Applications:

- Google Maps navigation agent uses GPS sensors and route-suggestion actuators guided by a shortest-time performance measure.
- Tesla Autopilot uses PEAS thinking with safety and comfort as performance measures and cameras/radar as sensors.
- iRobot Roomba is a classic simple agent example with dirt sensors and wheel/suction actuators.
- Amazon Alexa acts as a software agent, sensing via microphone and acting via speaker output and smart home actuators.
- Automated stock trading bots act as agents sensing market data and acting through buy/sell order actuators

KEY POINT BOX
• PEAS (Performance, Environment, Actuators, Sensors) is the single most repeated exam concept in this topic — practice PEAS for taxi, vacuum and one more example. • Agent function is abstract/infinite; Agent program is the real, finite, running code — a favourite comparison question. • Percept is a single input; Percept sequence is the full history — do not use these terms interchangeably in exams. • Always give both hardware (robot) and software (chatbot) examples of sensors/actuators to show full understanding. • Diagram of agent-environment interaction loop (sense-think-act) is expected in almost every answer on this topic.

EXPECTED QUESTIONS BOX
Q1. Define an intelligent agent. Explain the agent function and agent program with an example. (6 marks)
Q2. What is PEAS? Explain the PEAS description for an automated taxi driver agent. (8 marks)
Q3. Explain percepts, percept sequence, sensors and actuators with suitable examples. (6 marks)
Q4. Give the PEAS description for a vacuum cleaner agent and a medical diagnosis agent. (8 marks)
Q5. Draw and explain the generic structure of an intelligent agent interacting with its environment. (6 marks)

1.10 Agents and Environments

Agents and Environments describes the continuous interaction cycle in which an agent perceives its environment through sensors, processes the percept using its internal agent program, and acts upon the environment through actuators, with the environment's state changing as a result of that action.

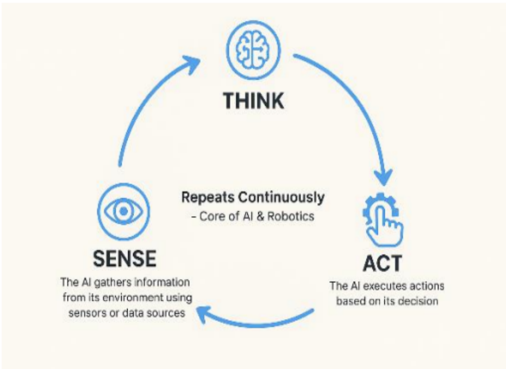
Introduction / Need:
An agent never exists in isolation. Its behaviour and success can only be judged in the context of the specific environment it operates in.

A self driving car agent, for example, is meaningless without describing the roads, traffic and weather it must handle — the agent and environment must always be studied together.

1.10.1 The Agent-Environment Interaction Loop

The interaction is cyclic: the agent perceives the current state of the environment, decides an action using its agent program, executes the action, and the environment changes state in response.

This cycle (sense, think, act) repeats continuously for as long as the agent is active, whether it runs for a fraction of a second (robot control loop) or over years (a long running monitoring agent).



1.10.2 Percept Sequence and the Agent Function Table

Since an agent's action can depend on its entire percept history, its behaviour can be represented as a lookup table mapping every possible percept sequence to an action.

This table quickly becomes impractically large for any non-trivial environment, so it is used mainly as a theoretical tool to define what an agent should do, not how to implement it.

1.10.3 The Vacuum World Example

The vacuum world (a simplified two-location environment used to illustrate agent behaviour) consists of two rooms — A and B — each of which can be clean or dirty, and an agent that can move left, move right, or suck dirt.

The agent perceives its current location and whether that location is dirty, and its performance measure is based on how much dirt is cleaned over a time period.

Sr. No.	Percept (Location, Status)	Action Taken by Agent
1	(A, Dirty)	Suck
2	(A, Clean)	Move Right
3	(B, Dirty)	Suck
4	(B, Clean)	Move Left
5	(A, Dirty),(A, Clean)	Move Right (based on percept sequence)

Advantages:

- Concrete visualisation — the agent-environment loop makes an abstract concept easy to trace step by step.
- Simplifies analysis — small examples like the vacuum world let students test agent logic without real hardware.
- Supports all agent types — the same loop framework applies to every type of agent studied later in this unit.
- Highlights feedback — showing the environment changing after each action teaches the importance of feedback in AI design.

Disadvantages:

- Table explosion — representing agent behaviour as a full percept-sequence table is infeasible for real environments.
- Oversimplified examples — the vacuum world does not capture the complexity of real world uncertainty and noise.
- Assumes clean percepts — the simple loop model often ignores sensor noise, which real agents must handle.

Applications:

- Household robotic vacuum cleaners like iRobot Roomba are directly modelled using the vacuum world abstraction.
- Thermostats use a simple sense-think-act loop to keep room temperature at a target value.
- Warehouse robots at Amazon fulfilment centres continuously sense shelf locations and act to move goods.
- Traffic signal control agents sense vehicle density and act by changing signal timings in smart cities.

KEY POINT BOX
• The sense-think-act loop is the backbone of every agent design — always draw this cycle in diagrams. • The Vacuum World is the standard SPPU example — memorise its PEAS and its percept-to-action table. • Full percept-sequence tables are theoretical only; real programs use compact, efficient decision logic. • Environment state changes because of both the agent's own action and other independent factors. • Do not confuse the Vacuum World with the general concept of 'Nature of Environments' — the vacuum world is just one example environment.

EXPECTED QUESTIONS BOX

SPPU EXPECTED QUESTIONS — Agents and Environments

Q1. Explain the agent-environment interaction cycle with a neat diagram. (6 marks)

Q2. Describe the vacuum cleaner world as an example of an agent and its environment. (8 marks)

Q3. What is a percept sequence? Explain its role in determining agent behaviour. (5 marks)

Q4. Why is a full agent function table impractical for real world environments? Explain. (5 marks)

1.11 Good Behavior: Concept of Rationality

Rationality is the property of an agent selecting, for every possible percept sequence, the action that is expected to maximise its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Introduction / Need:
Rationality gives AI a precise, testable definition of 'good behaviour', replacing vague ideas like 'the agent should act smart' with a measurable standard.

This concept lets engineers judge whether an agent is well designed, without needing the agent to behave exactly like a human or be all-knowing.

1.11.1 Rational Agent Definition:

A rational agent is one that does the right thing, meaning it chooses actions that lead to the best expected outcome given what it currently knows.

Rationality is not the same as perfection — a rational agent can still make a mistake if the outcome depends on information it did not and could not have.

1.11.2 Performance Measure

The performance measure (an objective criterion used to evaluate how successful an agent's behaviour is) must be defined in terms of what is wanted in the environment, not how the agent should behave internally.

A poorly designed performance measure can cause an agent to find loopholes, such as a vacuum cleaning agent that gets rewarded for cleanliness simply sweeping dirt under a rug instead of removing it.

1.11.3 Four Factors That Determine Rationality

Explanation:
According to Russell and Norvig, rationality at any given time depends on four things: the performance measure that defines the criterion of success, the agent's prior knowledge of the environment, the actions the agent can perform, and the agent's percept sequence to date.

For any given combination of these four factors, the rational action is the one expected to maximise the performance measure, based on the evidence provided by percepts and whatever knowledge the agent has.

Sr. No.	Factor	Meaning	Example (Vacuum Agent)
1	Performance measure	Defines what counts as success	Amount of dirt cleaned over time
2	Prior knowledge of environment	What the agent already knows	Knows there are exactly two rooms
3	Actions available	What the agent is capable of doing	Move left, move right, suck
4	Percept sequence to date	Everything perceived so far	Current room and dirt status observed

1.11.4 Rationality versus Omniscience, Learning and Autonomy

Omniscience (knowing the actual outcome of every action in advance) is not required for rationality — a rational agent only needs to act optimally based on what it currently knows, even if the real world outcome later turns out badly.

Rational agents benefit from information gathering (taking exploratory actions to reduce uncertainty before acting) since acting on incomplete or wrong information can lead to poor decisions.

Advantages:

- Objective standard — rationality gives a measurable, testable definition of good agent behaviour, unlike vague human comparisons.
- Fair evaluation — since rationality does not require omniscience, agents are judged fairly based only on available information.
- Encourages learning — the rationality framework naturally pushes toward agents that learn and improve rather than stay static.
- Encourages information gathering — rational agents are motivated to explore and reduce uncertainty before big decisions.
- Design guidance — the four rationality factors give engineers a clear checklist while designing any new agent.

Disadvantages:

- Performance measure difficulty — defining a correct, loophole-free performance measure is often surprisingly hard in practice.
- No guarantee of good outcomes — a rational agent can still fail in the real world due to lack of information, which can confuse newcomers.
- Computational cost — computing the truly best action for every percept sequence can be computationally expensive or infeasible.
- Overemphasis on optimality — real world agents often use 'good enough' bounded rational behaviour instead of true optimal rationality.

Applications:

- Self driving car companies like Waymo define detailed performance measures balancing safety, speed and comfort to guide rational behaviour.
- Recommendation systems on Netflix are designed with a performance measure based on user watch time and satisfaction.

- Automated trading agents use rationality principles to maximise expected profit given available market information.
- Search and rescue robots use information gathering actions to reduce uncertainty about a disaster environment before acting.
- Game playing AI like AlphaGo uses rational decision making to select moves that maximise expected win probability.

KEY POINT BOX
• Rationality = best expected action given performance measure, prior knowledge, actions and percept sequence — memorise all four factors exactly. • Rational does not mean omniscient or perfect — this distinction is the most common trick question in SPPU exams. • A badly designed performance measure can cause unwanted agent behaviour (reward hacking) — always mention the vacuum-sweep-under-rug example. • Learning and Autonomy both increase rationality over time by reducing reliance on fixed prior knowledge. • This topic connects directly to PEAS — the performance measure factor is the same as the P in PEAS.
EXPECTED QUESTIONS BOX
Q1. Define rationality. Explain the four factors that determine the rationality of an agent at given time. (8 M) **Q2.** Differentiate between rationality and omniscience with a suitable example. (6 marks) **Q3.** Explain the concept of performance measure with an example of a poorly designed one. (6 marks) **Q4.** How do learning and autonomy relate to the concept of rational agents? (5 marks) **Q5.** Explain what it means for an agent to behave rationally, with reference to a self driving car. (8 marks)

1.12 Types of Agents

Types of Agents refers to the classification of intelligent agents into five categories — Simple Reflex, Model-Based Reflex, Goal-Based, Utility-Based, and Learning agents — based on how much internal structure and reasoning capability each agent uses to select its actions.

Introduction / Need:

Not every problem needs a complex AI system. Some tasks can be solved by very simple agents, while others need deep planning and learning capability.

This classification, ordered roughly from simplest to most advanced, helps engineers pick the minimum agent design complexity that is sufficient to solve a given problem well.

1.12.1 Simple Reflex Agents

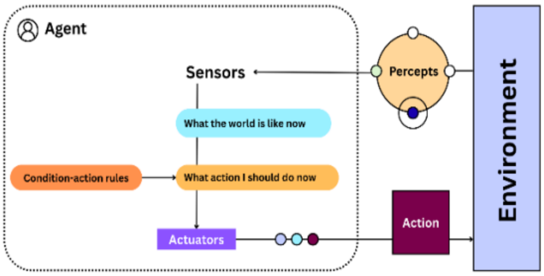
A simple reflex agent (an agent that selects actions based only on the current percept, ignoring the rest of the percept history) uses condition-action rules of the form 'if condition then action'.

It works well only in fully observable environments, since it has no memory and cannot handle situations where the correct action depends on what happened earlier.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules  // a set of condition-action rules
  state <- INTERPRET-INPUT(percept)
  rule <- RULE-MATCH(state, rules)
  action <- rule.ACTION
  return action
```

Explanation of Program:

This program interprets the raw percept into a state description, matches that state against a fixed set of condition-action rules, and returns the action linked to the matching rule, with no memory of past percepts used at all.



Simple Reflex Agents

1.12.2 Model-Based Reflex Agents

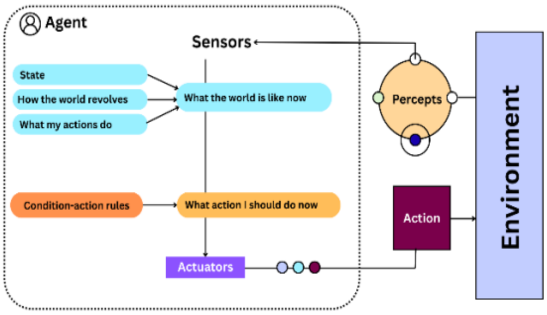
A model-based reflex agent (an agent that maintains an internal state to track the part of the world it cannot currently see) can handle partially observable environments better than a simple reflex agent.

It uses a model of the world (knowledge about how the environment evolves and how its own actions affect the environment) to update its internal state after every action.

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, model, rules, last_action
  state <- UPDATE-STATE(state, last_action, percept, model)
  rule <- RULE-MATCH(state, rules)
  action <- rule.ACTION
  last_action <- action
  return action
```

Explanation of Program:

The agent first updates its internal state using the previous state, its last action, the new percept, and its world model, then matches this richer state against condition-action rules to choose the next action.



Model-Based Reflex Agents

1.12.3 Goal-Based Agents

Explanation:

A goal-based agent (an agent that chooses actions by considering how they help reach an explicitly defined goal, not just the current state) can reason about the future consequences of actions.

It typically uses search and planning (looking ahead through possible action sequences to find one that reaches the goal) instead of simple fixed rules.

1.12.4 Utility-Based Agents

A utility-based agent (an agent that chooses actions based on a utility function measuring the desirability of different possible outcomes) can handle situations where multiple goals conflict or where goals are not equally important.

The utility function (a numeric measure representing how happy or satisfied the agent would be in a given state) allows the agent to select the action with the highest expected utility, balancing trade-offs like speed versus safety.

1.12.5 Learning Agents

A learning agent (an agent that can improve its own performance over time using feedback from its own experience) can operate effectively in environments that were not fully known when the agent was first designed.

It has four conceptual parts: the performance element (selects external actions, similar to earlier agent types), the learning element (improves the performance element using feedback), the critic (evaluates the agent's performance against a fixed standard), and the problem generator (suggests exploratory actions to discover new information).

Comparison Table:

Sr. No.	Agent Type	Decision Basis	Memory of Past	Limitation
1	Simple Reflex	Current percept only, fixed rules	No	Fails in partially observable environments
2	Model-Based Reflex	Current percept + internal state/model	Yes (internal state)	Still reactive, no explicit future planning
3	Goal-Based	Actions chosen to reach a defined goal	Yes	Can be slow due to search/planning cost
4	Utility-Based	Actions chosen to maximise a utility value	Yes	Designing a correct utility function is hard
5	Learning	Improves behaviour using experience/feedback	Yes (learned knowledge)	Needs time and data to reach good performance

WORKED EXAMPLE

Given: A vacuum cleaning agent operates in a two-room environment (Room A and Room B). It must decide its action based on its current percept only, following fixed rules: if the current room is dirty, suck; else move to the other room.

Find: Identify the type of agent this describes and justify the classification.

Solution:

Step 1: Observe that the agent's action depends only on the current percept (room and dirt status), not on any stored history.

Step 2: Check that the agent uses fixed condition-action rules ('if dirty then suck, else move') rather than any goal or utility calculation.

Step 3: Confirm the agent keeps no internal state describing rooms it currently cannot see.

Step 4: Compare this behaviour against the five agent type definitions listed above.

Step 5: Match the behaviour to the definition of an agent that reacts using only the current percept and fixed rules.

Ans: This is a Simple Reflex Agent, since it selects its action using only the current percept and a fixed set of condition-action rules, with no memory or planning involved.

Advantages:

- Scales to problem difficulty — simple problems can use simple reflex agents, saving development effort compared to complex learning agents.
- Clear design roadmap — the five-type classification gives engineers a structured path to increase agent capability as needed.
- Handles uncertainty well — utility-based and learning agents can make good decisions even in unpredictable environments.
- Encourages long-term improvement — learning agents naturally adapt and improve without manual redesign.
- Flexible goal handling — goal-based and utility-based agents adjust automatically when objectives change.

Disadvantages:

- Reflex agents are fragile — simple and model-based reflex agents fail badly outside the exact conditions they were designed for.
- Planning cost — goal-based agents can be computationally expensive when the search space of possible actions is large.
- Utility design difficulty — creating an accurate utility function that correctly balances multiple competing objectives is hard.
- Learning agents need data and time — they may perform poorly early on, before enough feedback has been collected.

Applications:

- Simple Reflex Agent — automatic room light switches that turn on/off purely based on current motion sensor input.
- Model-Based Reflex Agent — modern car adaptive cruise control tracking nearby vehicles using an internal world model.
- Goal-Based Agent — GPS navigation systems like Google Maps that plan a route to reach a specified destination goal.
- Utility-Based Agent — ride-sharing apps like Uber that balance price, time and driver distance using a utility calculation.
- Learning Agent — Netflix's recommendation engine that continuously improves suggestions based on user watch feedback.

KEY POINT BOX
• Five agent types, in order of increasing capability: Simple Reflex, Model-Based Reflex, Goal-Based, Utility-Based, Learning — this exact order is commonly tested. • Simple Reflex has no memory; Model-Based Reflex adds internal state — this is the most common comparison question. • Goal-Based asks 'does this help reach my goal'; Utility-Based asks 'how good is this outcome numerically' — do not mix these up. • Learning Agent has four parts: Performance element, Learning element, Critic, Problem generator — memorise all four for full marks. • Always be ready to classify a described scenario into one of the five types — this is a guaranteed applied question.

EXPECTED QUESTIONS BOX
Q1. Explain the five types of intelligent agents with neat diagrams. (8 marks)
Q2. Differentiate between Simple Reflex Agent and Model-Based Reflex Agent with examples. (6 marks)
Q3. Explain the structure of a Learning Agent with its four main components. (8 marks)
Q4. Compare Goal-Based Agent and Utility-Based Agent with suitable examples. (6 marks)
Q5. For a given scenario, identify the type of intelligent agent and justify your answer. (5 marks)

1.13 Nature of Environments

The nature of an environment refers to the set of properties — observability, number of agents, determinism, episodic or sequential structure, dynamics, continuity, and prior knowledge — used to classify the task environment in which an intelligent agent operates, which strongly influences the design of the agent program.

Introduction / Need:
The same agent design that works perfectly in one environment can fail completely in another, so classifying environments is a necessary first step before designing any agent.

Russell and Norvig define a standard set of environment properties that let engineers describe any task environment precisely and choose an appropriate agent architecture for it.

1.13.1 PEAS-Based Environment Specification

Before classifying environment properties, the environment must first be specified using PEAS (Performance, Environment, Actuators, Sensors), as covered in the Intelligent Agents topic.

Once the PEAS description is ready, each environment property below is applied to describe how difficult or complex that environment is for an agent to handle.

1.13.2 Key Properties of Task Environments

Fully Observable vs Partially Observable: an environment is fully observable (the agent's sensors give it complete access to the relevant environment state at every point) or partially observable (sensors give incomplete or noisy information), such as chess (fully observable) versus poker (partially observable).

Single-Agent vs Multi-Agent: an environment involves only one agent (single-agent, such as a crossword puzzle solver) or multiple agents that may cooperate or compete (multi-agent, such as chess or a multiplayer game).

Sr. No.	Property	Easier Case	Harder Case	Example (Harder Case)
1	Observability	Fully Observable	Partially Observable	Poker (opponent cards hidden)
2	Number of Agents	Single-Agent	Multi-Agent	Chess (competitive multi-agent)
3	Determinism	Deterministic	Stochastic	Self-driving car (uncertain traffic)
4	Episode Structure	Episodic	Sequential	Chess (each move affects future moves)
5	Dynamics	Static	Dynamic	Taxi driving (environment keeps changing)

1.13.3 Applying Environment Properties: Automated Taxi Example

Explanation:

The automated taxi environment can be classified across all seven properties, showing how a single realistic environment can be difficult on almost every dimension at once.

It is partially observable (cannot see everything on the road), multi-agent (other drivers, pedestrians), stochastic (unpredictable traffic behaviour), sequential (each driving decision affects future ones), dynamic (traffic changes continuously), continuous (steering and speed are continuous values), and effectively unknown in some regions (new road layouts).

WORKED EXAMPLE
Given: Classify the environment of a Chess Game playing agent (with a visible board) using the standard environment properties. **Find:** Determine whether the chess environment is observable, single/multi-agent, deterministic, episodic/sequential, static/dynamic and discrete/continuous. **Solution:** **Step 1:** Check observability: since both players can see the full board at all times, the environment is Fully Observable. **Step 2:** Check number of agents: since two players compete against each other, the environment is Multi-Agent. **Step 3:** Check determinism: since a chess move always produces the same predictable resulting position, the environment is Deterministic. **Step 4:** Check episode structure: since each move affects the position for all future moves, the environment is Sequential. **Step 5:** Check dynamics: since the board does not change while a player is thinking, the environment is Static. **Step 6:** Check state/action space: since there is a finite, countable number of board positions and legal moves, the environment is Discrete. **Ans: Chess is Fully Observable, Multi-Agent, Deterministic, Sequential, Static and Discrete.**

Advantages:

- Guides agent design — knowing environment properties tells engineers whether a simple reflex agent is enough or a learning agent is required.
- Improves system evaluation — environment classification allows fair comparison between agents designed for similar problems.
- Highlights real complexity — properties like partial observability explain why some AI problems (e.g. poker) remain harder than others (e.g. chess).
- Supports risk assessment — dynamic and stochastic classification helps identify environments needing more safety testing, like self-driving.
- Universal applicability — the same seven properties can classify any environment, from board games to robotics to trading.

Disadvantages:

- Simplified categories — many real environments fall between the two extremes (e.g. 'mostly observable') and are not purely binary.
- Subjective boundaries — deciding whether an environment counts as dynamic or static can depend on the time scale considered.
- Does not measure difficulty directly — classification tells the type of challenge but not exactly how hard the environment is to solve.
- Overlapping properties — properties like stochastic and partially observable often interact, making pure classification harder in practice.

Applications:

- Self-driving cars are classified as partially observable, stochastic, dynamic environments requiring advanced perception and planning.
- Chess engines like Stockfish operate in a fully observable, deterministic, discrete environment, allowing deep tree search.
- Poker playing AI like Pluribus is designed for a partially observable, stochastic, multi-agent environment.
- Warehouse robots at Amazon operate in a mostly known, discrete, dynamic environment shared with other robots.
- Weather prediction AI systems handle a continuous, stochastic, partially observable natural environment.

KEY POINT BOX

- Seven properties to memorise exactly: Observability, Number of Agents, Determinism, Episodic/Sequential, Static/Dynamic, Discrete/Continuous, Known/Unknown.
- Chess vs Poker is the most common exam comparison for observability (fully vs partially observable).
- Automated Taxi is the standard SPPU example showing an environment that is hard on almost every property.
- Sequential means past actions affect future decisions; Episodic means each decision is independent — a frequent confusion point.
- Always classify a given scenario across all seven properties in exam answers, not just one or two.

EXPECTED QUESTIONS BOX

Q1. Explain the different properties used to classify task environments. (8 marks)

Q2. Differentiate between fully observable and partially observable environments with examples. (5 marks)

Q3. Classify the environment of an automated taxi driver across all environment properties. (8 marks)

Q4. Explain deterministic vs stochastic and episodic vs sequential environments with examples. (6 marks)

Q5. Why is designing an agent for a dynamic, partially observable environment more difficult? Explain. (6 marks)

1.14 Structure of Agents

The structure of an agent describes how an agent is built by combining an architecture (the physical or computational platform on which the agent runs, such as sensors, actuators and a computing device) with an agent program (the software that implements the agent function), following the relationship Agent = Architecture + Program.

Introduction / Need:
So far this unit has described what agents do (their function) and how they are classified (their type). This topic explains how an agent is actually constructed internally.

Understanding agent structure connects the theoretical PEAS and agent-type ideas to a concrete, implementable design that a computer engineer can actually build.

1.14.1 Agent = Architecture + Program

The architecture (the underlying computing device or robotic hardware, including sensors and actuators, that the agent program runs on) provides the physical means for perceiving and acting.

The agent program (the software implementing the mapping from percepts to actions) is what actually contains the decision making logic.

1.14.2 Skeleton Structure of an Agent Program

A general agent program continuously receives percepts, updates its knowledge or internal state if needed, chooses an action using its decision logic, and sends that action to the actuators.

This same skeleton structure underlies all five agent types studied earlier, from the simplest reflex agent to the most advanced learning agent, differing only in how much internal reasoning happens inside the loop.

```
function AGENT(percept) returns an action
  persistent: internal_state  // memory of the world, if any
  internal_state <- UPDATE-STATE(internal_state, percept)
  action <- CHOOSE-BEST-ACTION(internal_state)
  return action
```

Explanation of Program:
This general skeleton updates the agent's internal state using the new percept, then chooses an action based on that internal state, and this same basic loop is specialised differently for each of the five agent types studied earlier.

1.14.3 Table-Driven Agent and Its Limitations

Explanation:
A table-driven agent (an agent whose agent program is implemented as a giant lookup table mapping every possible percept sequence directly to an action) is the most literal but least practical way to implement an agent.

This approach is only of theoretical interest, because the table needed to cover all possible percept sequences in any real environment would be astronomically large and impossible to build or store.

Sr. No.	Agent Program Style	How Action is Chosen	Practicality
1	Table-driven	Direct lookup of percept sequence in a table	Impractical, table too large for real problems
2	Simple reflex program	Condition-action rule matching on current percept	Practical for simple, fully observable tasks
3	Model-based program	Rule matching on updated internal state	Practical for partially observable tasks
4	Goal/Utility-based program	Search or evaluation over possible action sequences	Practical but computationally heavier
5	Learning program	Rules/model updated automatically from feedback	Practical for unknown or changing environments

Advantages:

- Clarifies implementation — separating architecture from program clarifies exactly what hardware and software an engineer must build.
- Reusable architecture — the same physical robot architecture can run different agent programs for different tasks.
- Explains agent type differences — the skeleton structure shows that all five agent types are variations of one common loop.
- Highlights practical limits — the table-driven agent example clearly shows why compact decision logic is necessary in practice.

Disadvantages:

- Still abstract for beginners — students can find the difference between architecture and program confusing without concrete examples.
- Skeleton hides real complexity — the simple loop pseudocode does not show the real difficulty of building CHOOSE-BEST-ACTION for hard problems.
- Architecture constraints — a poor or limited hardware architecture (few sensors) can restrict what any agent program can ever achieve.

Applications:

- Roomba robotic vacuum cleaners combine a fixed hardware architecture (wheels, dirt sensor) with a simple reflex agent program.
- Self-driving car platforms like Tesla combine sensor/actuator architecture with a continuously updated learning agent program.
- Chatbot platforms combine a server architecture with an NLP-based goal or utility driven agent program.
- Industrial robotic arms combine fixed mechanical architecture with model-based agent programs for precise assembly tasks.

KEY POINT BOX
• Agent = Architecture + Program is the exact formula expected in SPPU answers — always state it explicitly. • Architecture is the hardware/platform; Program is the decision-making software — this pairing is a common short-answer question. • Table-driven agent is a theoretical worst case used to justify why compact reflex, goal, utility and learning programs are needed. • The general agent skeleton (update state, choose action) is common to all five agent types from the Types of Agents topic. • Always connect this topic back to PEAS and Types of Agents in longer answers, since SPPU often asks combined questions.
EXPECTED QUESTIONS BOX
Q1. Explain the relationship between agent architecture and agent program with a diagram. (6 marks) **Q2.** What is a table-driven agent? Explain why it is impractical for real environments. (6 marks) **Q3.** Describe the general skeleton structure of an agent program. (5 marks) **Q4.** Explain how the structure of an agent relates to the different types of agents studied. (8 marks)

Unit 1 Summary

SUMMARY BOX
UNIT 1 SUMMARY • 1.1 Introduction to AI — AI is defined via four approaches: thinking humanly, thinking rationally, acting humanly, acting rationally. • 1.2 Foundations of AI — AI draws from Philosophy, Mathematics, Economics, Neuroscience, Psychology, Linguistics and Computer Engineering. • 1.3 History of AI — Born at the 1956 Dartmouth Conference; passed through two AI winters before the modern deep learning boom (2012 onward). • 1.4 Limits of AI — Common sense reasoning, data dependency, poor generalisation and explainability (black box) remain unsolved technical limits. • 1.5 Ethics of AI — Bias, privacy, accountability, job displacement and safety/alignment are the core ethical concerns. • 1.6 Future of AI — AGI, Generative AI, Edge AI, regulation and human-AI collaboration define upcoming trends. • 1.7 AI Components — Knowledge representation, reasoning, learning, perception, planning and communication form every AI system. • 1.8 AI Architectures — Symbolic (explainable rules), Connectionist (learned neural weights), and Hybrid combine both. • 1.9 Intelligent Agents — An agent perceives via sensors and acts via actuators; PEAS (Performance, Environment, Actuators, Sensors) defines any task environment. • 1.10 Agents and Environments — The sense-think-act loop connects an agent to its environment; the Vacuum World is the standard example. • Important formula/definition — Rational action = action that maximises expected performance measure given percept sequence and prior knowledge. • 1.12 Types of Agents — Simple Reflex, Model-Based Reflex, Goal-Based, Utility-Based and Learning agents, in increasing order of capability. • 1.13 Nature of Environments — Seven properties: Observable, Number of Agents, Deterministic, Episodic/Sequential, Static/Dynamic, Discrete/Continuous, Known/Unknown. • 1.14 Structure of Agents — Agent = Architecture + Program; table-driven agents are theoretically valid but practically impossible to build. • Most repeated SPPU exam question — 'Explain PEAS with the example of an automated taxi driver agent' appears almost every year.